

《网络应用架构设计与开发》 课程设计报告

说明：请将封面信息替换为本人学号与姓名；截图位置已用【图 X-X】标注，导出 PDF 前请插入实际页面截图；在线地址与代码仓库请填写真实链接。

项目	内容
学号	【202330450191】
姓名	【陈焕林】
设计题目	基于大数据分析的电子商务购物网站
网站类型	电子商务网站（购物类）
代码仓库	【请填写 GitHub/Gitee 地址】
在线演示	【请填写部署 URL，有效期至 2026-07-01】
提交日期	2026 年 5 月

目录

- 1. 一、设计题目概述
- 2. 二、需求分析报告
- 3. 三、系统设计报告
- 4. 四、系统实现报告
- 5. 五、测试报告
- 6. 六、附录

一、设计题目概述

在课程实验基础上，本项目实现了一套 **B2C 电子商务购物网站**，在完整购物流程（注册登录 → 浏览商品 → 购物车 → 下单支付 → 订单管理）之上，按课程要求扩展了 **大数据采集、数据分析、商品推荐** 及选做功能 **反爬虫** 侦测与应对。

1.1 网站类型

电子商务网站（购物类）

1.2 核心功能要求对照

课程要求	本项目实现情况
用户注册/登录	✓ /api/auth/register、/api/auth/login，JWT 鉴权
商品展示	✓ 首页列表、分类筛选、搜索、商品详情

课程要求	本项目实现情况
订购/收费	✅ 购物车、结算、模拟支付、订单状态流转
三类角色	✅ 用户（customer）、销售人员（seller）、管理者（admin）
大数据采集（3.2）	✅ 登录/浏览/购买/操作四类日志落库
数据分析与推荐（3.3）	✅ 用户画像、趋势预测、异常监控、排行榜、趋势图、两种推荐
附加功能（3.4）	✅ 反爬虫侦测与应对（限流、蜜罐、人机验证）

1.3 技术栈总览

层次	技术选型
前端	Vue 3 + Vite + Vue Router + Pinia + Element Plus + Axios
后端	Node.js + Express
数据库	PostgreSQL + Prisma ORM
缓存/限流	Redis（可选，未启动时内存兜底）
对象存储	阿里云 OSS（商品图片）
认证	JWT + bcrypt 密码加密

二、需求分析报告

2.1 Step 1：业务场景与痛点分析

2.1.1 业务场景

典型 B2C 电商场景：平台方（管理者）管理卖家与全局报表；卖家维护商品目录与订单；消费者在站内完成浏览与购买。课程要求在此基础上引入 **大数据能力**，使平台不仅能“卖货”，还能 **采集行为数据 → 分析 → 预测 → 推荐 → 防护异常爬取**。

2.1.2 痛点（需解决的课题）

编号	痛点	影响
P1	缺乏统一的用户行为数据沉淀	无法做画像与推荐
P2	销售趋势与异常难以人工发现	运营响应滞后
P3	商品信息易被爬虫批量抓取	带宽浪费、数据被竞品利用
P4	多角色权限边界不清	安全风险与误操作
P5	图片与订单并发场景下数据一致性	超卖、支付状态错误

2.2 Step 2：可能的解决方案

痛点	方案 A	方案 B
P1	埋点 SDK + 消息队列异步写入	应用层服务同步写 PostgreSQL 日志表
P2	外接 BI 平台	自研统计服务 + 管理后台可视化
P3	WAF 云服务	应用层频率限制 + UA 检测 + 蜜罐 + 人机验证
P4	RBAC 中间件	JWT 载荷 role + 路由级 requireAdmin/requireSeller
P5	数据库事务	Prisma \$transaction 扣库存与写订单

2.3 Step 3：可行性分析

综合考虑 个人项目工期、部署成本、课程验收可演示性，采用：

- **方案选择**：关系型库存日志 + Node 统计服务 + Redis/内存限流（不引入过重的大数据组件如 Flink/Kafka）。
- **可行性**：Prisma 迁移与种子脚本可快速复现演示数据；前后端分离便于分模块开发与测试。
- **风险**：数据量极大时 PostgreSQL 聚合变慢 → 可通过索引、按日汇总表扩展（见系统设计应急预案）。

2.4 Step 4：实施方案、技术指标与进度

2.4.1 功能需求清单（任务清单）

3.1 用户角色定义

 用户（Customer）

功能	说明
注册/登录/注销	自主注册，JWT 维持会话
浏览商品	列表、搜索、分类、详情（需登录访问前台页面）
购物流程	加购 → 结算 → 模拟支付 → 订单列表/详情
行为采集	浏览商品时上报停留时长（POST /api/analytics/browse）

 销售人员（Sales）

功能	说明
类别管理	添加/删除商品分类
商品管理	增删改商品、价格库存、上下架
订单处理	查看本店相关订单、发货等
运营数据	销售指标、操作/浏览日志查看

 管理者（Admin）

功能	说明
卖家管理	添加/删除卖家、重置密码
业绩监控	销售业绩、报表（按类别等维度）
大数据看板	采集概览、用户画像、趋势/异常/排行/图表
反爬虫监控	拦截事件、IP 解封

3.2 数据采集要求（大数据）

数据类型	采集内容	存储模型
用户-登录	时间、IP	LoginLog
用户-浏览	商品类别、停留秒数	BrowseBehaviorLog
用户-购买	类别、日期、单价、数量	PurchaseRecord
销售/管理-登录	时间、IP	LoginLog（共用）
销售/管理-操作	操作时间、内容、IP、账号	OperationLog

3.3 数据分析与推荐系统

类型	功能	实现模块
必做	用户画像（地域、购买力、偏好分类）	userProfileService
必做	销售趋势预测与评估	salesTrendService
必做	销售异常判别与监控	salesAnomalyService
必做	商品销售排行榜	salesRankingService
必做	销售趋势图（日/周/月）	getSalesTrendChart + SalesTrendChart.vue
必做	简单推荐（也买了...）	getAlsoBought
必做	协同过滤推荐	基于用户购买集合的 Jaccard 相似度

3.4 附加功能（选做）

功能	实现要点
反爬虫侦测与应对	频率/突发/UA/蜜罐；429 + 算术验证 + IP 临时封禁

2.4.2 非功能技术指标

指标	目标值
接口响应（列表查询）	P95 < 500ms（本地演示环境）
并发用户（课程演示）	≥ 50 模拟请求不崩溃

指标	目标值
反爬阈值	未登录约 35 次/分钟/IP；突发 12 次/10 秒
数据一致性	下单扣库存同一事务
安全	密码 bcrypt；管理接口 JWT + 角色校验

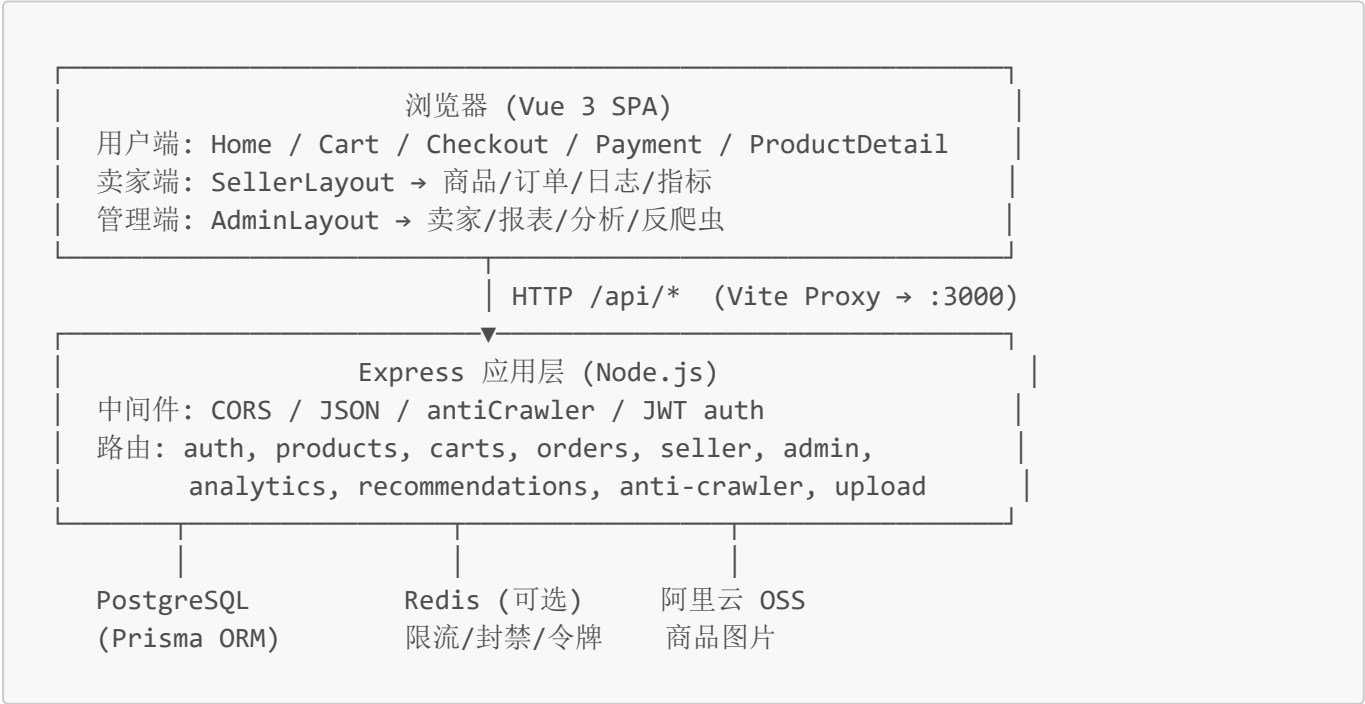
2.4.3 开发规划（示例甘特）

阶段	周期	内容
第 1 周	需求与设计	角色划分、ER 图、接口清单
第 2 周	核心交易	商品、购物车、订单、支付
第 3 周	卖家/管理后台	权限、报表、卖家 CRUD
第 4 周	大数据与推荐	采集、画像、趋势、推荐
第 5 周	选做+测试+部署	反爬虫、种子数据、联调、上线

三、系统设计报告

3.1 整体架构设计

采用 前后端分离 + RESTful API 架构。

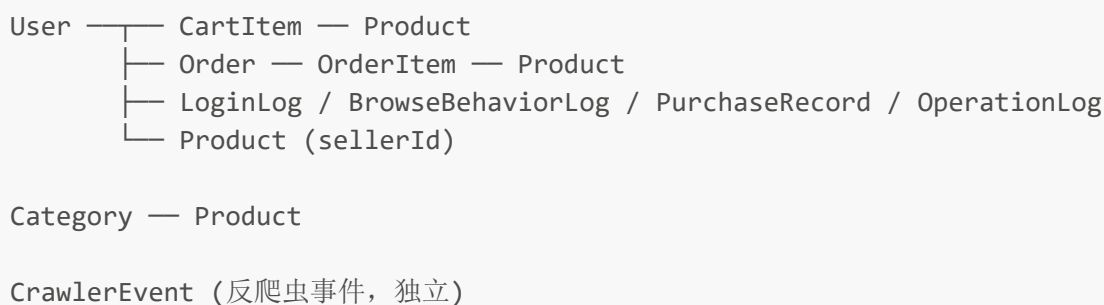


3.1.1 部署架构（建议）

组件	建议部署
前端静态资源	阿里云 OSS + CDN 或 Nginx

组件	建议部署
后端 API	云服务器 ECS / 轻量应用服务器
PostgreSQL	云数据库 RDS
Redis	云 Redis 或同机部署

3.2 数据库设计（核心实体）



角色枚举: `admin` | `seller` | `customer`

3.3 模块设计

3.3.1 用户与认证模块

项目	说明
功能	注册、登录、获取当前用户
技术	<code>bcryptjs</code> 哈希密码; <code>jsonwebtoken</code> 签发 24h Token
接口	<code>POST /api/auth/login</code> 、 <code>POST /api/auth/register</code> 、 <code>GET /api/auth/me</code>

3.3.2 商品与交易模块

子模块	职责
商品服务	分页列表、搜索、详情、分类
购物车	增删改数量、合并登录用户购物车
订单服务	创建订单（事务扣库存）、支付、取消、确认收货
上传服务	图片压缩（sharp）→ OSS

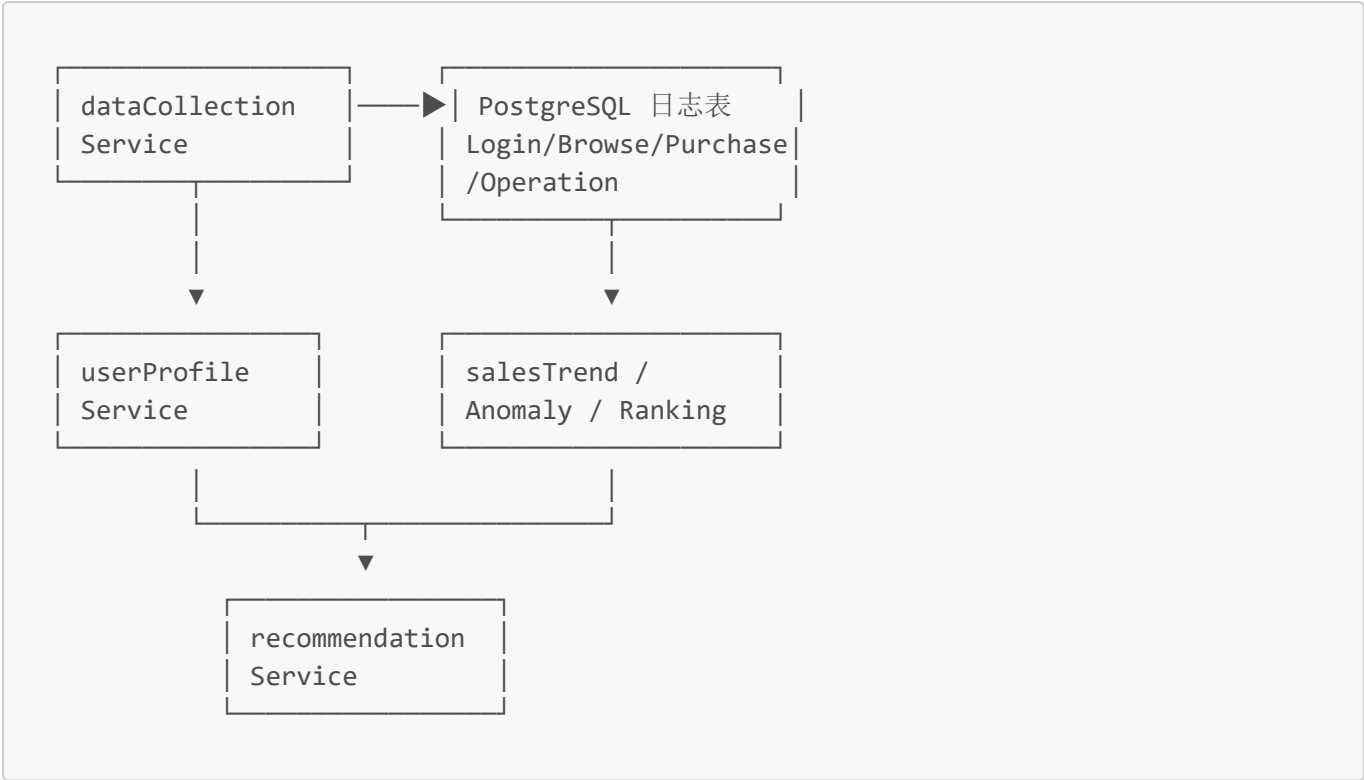
3.3.3 卖家模块

- 路由前缀: `/api/seller/*`，中间件校验 `seller` 或 `admin` 角色。
- 商品与分类 CRUD 均绑定 `sellerId`，实现数据隔离。

3.3.4 管理模块

- 路由前缀： `/api/admin/`， `requireAdmin`。
- 卖家管理、销售报表、分析看板、反爬虫管理。

3.3.5 大数据模块（课程重点）



用户画像算法概要：

- **地域**：优先解析订单收货地址；否则根据登录 IP 映射省份（`regionParser`）。
- **购买力**：按累计消费与订单数划分高/中/低/潜在。
- **偏好分类**：浏览停留时长 + 购买金额加权打分，取 Top5 类别。

销售趋势预测：对近 N 日销售额序列做滑动平均 / 线性外推，输出未来数日预测值及评估指标（MAE 等）。

异常监控：按日/周聚合销售额，与历史均值比较，偏离超过阈值标记异常。

协同过滤：构建用户-商品购买集合，计算用户间 Jaccard 相似度，聚合邻居购买商品得分排序推荐。

3.3.6 反爬虫模块

环节	设计
保护范围	<code>GET /api/products/</code> 、 <code>GET /api/recommendations/</code>
侦测	滑动窗口频率、10s 突发、可疑 User-Agent、蜜罐路径
应对	429 + <code>needVerify</code> ；算术挑战换 <code>X-Crawler-Token</code> ；多次违规 Redis/内存封禁 IP
审计	<code>CrawlerEvent</code> 表 + 管理后台查询/解封

蜜罐路径（不对前端暴露）：

- `GET /api/products/export-all-secret`
- `GET /api/products/catalog.json`

3.4 接口设计（节选）

方法	路径	说明
POST	/api/auth/login	登录
GET	/api/products	商品列表
POST	/api/orders	创建订单
POST	/api/orders/:id/pay	模拟支付
POST	/api/analytics/browse	上报浏览
GET	/api/recommendations/also-bought/:id	关联推荐
GET	/api/recommendations/for-you	协同过滤（需登录）
GET	/api/admin/analytics/user-profiles	用户画像列表
GET	/api/admin/analytics/sales-trend-chart	趋势图数据
GET	/api/anti-crawler/challenge	获取验证题
POST	/api/anti-crawler/verify	提交验证

3.5 前端路由设计

路径	角色	页面
/home	用户	商品首页 + 推荐
/product/:id	用户	详情 + 浏览埋点 + 也买了
/cart、/checkout、/order/:id/pay	用户	交易链路
/seller/*	卖家	商品/订单/日志
/admin/*	管理员	仪表盘、分析、反爬虫

3.6 风险评估与应急预案（PM 视角）

风险	可能性	影响	应急预案	可行性
PostgreSQL 连接失败	中	全站不可用	.env 检查；Docker 一键启动；健康检查 /health	高
OSS 配置错误导致图片 404	中	商品无图	种子数据使用已上传 URL；后台重新上传	高
Redis 未启动	高	限流精度下降	服务自动降级为内存 Map，单实例演示可用	高
支付接口 400（日志 userId 错误）	低	无法付款	事务内使用 order.userId；单测支付流程	已修复

风险	可能性	影响	应急预案	可行性
爬虫误判正常用户	中	体验差	提高已登录用户阈值；前端自动弹窗验证；后台解封 IP	高
日志表数据膨胀	低	查询变慢	按 <code>createdAt</code> 索引；定期归档；后续可加日汇总表	中
JWT 泄露	低	账号被盗	HTTPS 部署；短过期时间；敏感操作二次校验	中

四、系统实现报告

4.1 开发环境与工具

类别	版本/工具
操作系统	Windows 10/11
Node.js	建议 18+
包管理	npm
IDE	Visual Studio Code / Cursor
数据库	PostgreSQL（本地端口示例 5433）
可选	Redis 6+、阿里云 OSS 账号
AI 辅助	Cursor（Agent 模式）

4.1.1 项目目录结构

```
shopping/
├── back/                                # 后端
│   ├── prisma/                         # Schema 与迁移
│   ├── scripts/                        # seed、图片上传、反爬测试
│   └── src/
│       ├── controllers/
│       ├── middleware/                 # auth、antiCrawler
│       ├── routes/
│       ├── services/                  # 业务与大数据服务
│       └── utils/
├── front/                              # 前端 Vue3
│   └── src/
│       ├── views/                     # 页面
│       ├── layouts/                   # Admin/Seller 布局
│       └── store/                      # Pinia
```

```
|      | — utils/      # request、crawlerGuard  
|      | — 课程设计报告.md
```

4.2 本地运行与部署步骤

4.2.1 环境变量（`back/.env` 示例）

```
DATABASE_URL="postgresql://user:pass@localhost:5433/shopping_db"  
JWT_SECRET=your_jwt_secret  
PORT=3000  
  
OSS_REGION=oss-cn-guangzhou  
OSS_ACCESS_KEY_ID=***  
OSS_ACCESS_KEY_SECRET=***  
OSS_BUCKET=your-bucket  
OSS_ENDPOINT=oss-cn-guangzhou.aliyuncs.com  
  
CRAWLER_MAX_REQUESTS=35  
CRAWLER_BURST_MAX=12  
CRAWLER_BAN_SEC=900
```

4.2.2 本地启动

```
# 1. 后端  
cd back  
npm install  
npx prisma migrate deploy  
npx prisma generate  
node scripts/seed.js      # 初始化测试数据  
npm run dev               # http://localhost:3000  
  
# 2. 前端  
cd front  
npm install  
npm run dev               # http://localhost:3001  
  
# 3. （可选）Redis  
redis-server  
  
# 4. 反爬虫演示脚本  
cd back  
node scripts/test-anti-crawler.js
```

4.2.3 生产部署要点

1. 构建前端： `cd front && npm run build`，将 `dist/` 部署至 Nginx/OSS。
2. 配置反向代理： `/api` → 后端 3000；开启 HTTPS。

- 3. 后端进程守护：`pm2 start src/app.js`。
- 4. 数据库迁移：生产库执行 `npx prisma migrate deploy`。
- 5. 填写报告：在线 URL、测试账号、仓库地址。

4.3 核心功能实现说明

4.3.1 购物流程

- 1. 用户登录后浏览 `/home`，请求 `GET /api/products`。
- 2. 进入详情页触发 `POST /api/analytics/browse` 记录类别与停留时间。
- 3. 加入购物车 `POST /api/carts`，结算页创建订单 `POST /api/orders`（事务扣减库存）。
- 4. 支付页 `POST /api/orders/:id/pay` 更新状态为 `paid` 并写入 `PurchaseRecord`。

4.3.2 数据采集实现

在 `dataCollectionService.js` 中封装：

- `recordLogin`：登录成功时写入 `LoginLog`。
- `recordBrowse`：详情页离开或定时上报。
- `recordPurchase`：支付成功后按订单项写入 `PurchaseRecord`。
- `recordOperation`：卖家/管理员关键操作写入 `OperationLog`。

4.3.3 管理端分析页面

页面路径	功能
<code>/admin/analytics/collection</code>	采集概览与最近日志
<code>/admin/analytics/user-profiles</code>	用户画像列表与详情
<code>/admin/analytics/sales-trend</code>	趋势预测
<code>/admin/analytics/sales-trend-chart</code>	日/周/月 SVG 趋势图
<code>/admin/analytics/sales-anomalies</code>	异常列表
<code>/admin/analytics/sales-ranking</code>	排行榜
<code>/admin/anti-crawler</code>	反爬虫监控

4.3.4 推荐系统实现

- **也买了**：查询购买/浏览过该商品的用户 → 统计其购买的其他商品频次排序。
- **猜你喜欢**：当前用户购买集合 vs 其他用户集合 → Jaccard → 加权推荐得分。
- **首页**：登录用协同过滤，未登录用热销 fallback。

4.3.5 反爬虫实现

- 中间件：`middleware/antiCrawler.js` 全局挂载，对保护路径调用 `antiCrawlerService.inspectRequest`。
- 前端：`utils/crawlerGuard.js` + `request.js` 拦截 429 自动弹窗验证并重试。
- 管理：`AntiCrawler.vue` 展示统计与解封。

4.4 第三方框架说明

框架/库	用途	说明
Vue 3	前端 UI 框架	课程允许，已在本报告中说明
Element Plus	组件库	表格、表单、布局
Express	HTTP 服务	路由与中间件
Prisma	ORM	类型安全访问 PostgreSQL
ali-oss	对象存储	商品图片 CDN
sharp	图片处理	上传前压缩

4.5 项目反思（PM 视角）

1. **预估准确性：**

大数据模块实际耗时高于预期，主因是聚合 SQL 与前端图表联调；通过种子脚本生成 160+ 订单样本后，分析页面才有可展示曲线。
2. **预案有效性：**

Redis 降级内存方案在本地未装 Redis 时验证了反爬流程，符合设计预期。
3. **心得体会：**

先固化 **日志表结构** 再开发分析接口，可避免返工；角色权限宜在路由层一次性拦截，而非每个 Controller 重复判断。
4. **改进方向：**

可增加日汇总表提升报表性能；推荐可引入 ItemCF；反爬可对接滑动验证码服务。

4.6 AI 工具使用记录（独立章节）

(1) AI 工具使用清单

项目	内容
使用工具	Cursor （Composer / Agent 模式）
使用频率	约占编码时间 40%~50% （骨架生成、Bug 修复、报告起草）
主要用途	代码生成、接口联调排错、Prisma 迁移、Markdown 报告、反爬模块实现

(2) 效率提升分析

项目	估计
节省时间	约 25~35 小时 （相对纯手写）
AI 效率最高	CRUD 路由、Vue 表格页模板、重复性 Service、种子脚本、接口 404 排查
AI 帮助有限	支付事务边界条件、OSS 密钥环境差异、Prisma 迁移与本地库端口不一致

(3) 问题案例（≥3 个）

案例 1：管理后台白屏 **useRouter is not defined**

【问题描述】
打开 `/admin/dashboard` 页面空白，控制台报错 **useRouter is not defined**。

【AI 的建议】

在 `<script setup>` 中补充 `import { useRouter } from 'vue-router'` 并实例化 `const router = useRouter()`。

【实际采纳情况】

☒ 完全采纳

【最终解决方案】

在 `Dashboard.vue` 增加缺失的 `import`，页面恢复正常。

【反思】

组合式 API 必须显式导入组合函数，模板中用了 `router.push` 就要检查 `script` 是否声明。

案例 2：支付接口返回 400**【问题描述】**

`POST /api/orders/:id/pay` 返回 400，前端提示支付失败。

【AI 的建议】

检查 `processSimulatedPayment` 中 `userActivityLog.create` 的 `userId`；查询订单时 `user` 未 `select id`；移除随机失败逻辑；前端展示 `response.data.message`。

【实际采纳情况】

☒ 完全采纳

【最终解决方案】

改为 `userId: order.userId`；去掉 10% 随机失败；完善 `Payment.vue` 错误提示。

【反思】

事务内写关联表要使用外键字段，不能假设 `include` 了所有字段。

案例 3：反爬虫误判与 Redis 未连接**【问题描述】**

本地未启动 Redis 时代码报错；需保证限流逻辑仍可用。

【AI 的建议】

封装 `storeGet/storeSet/storeIncr`，Redis 不可用时降级到 `Map` 内存存储；管理端提供 IP 解封。

【实际采纳情况】

☒ 完全采纳

【最终解决方案】

`antiCrawlerService.js` 实现双存储；`test-anti-crawler.js` 用于演示拦截。

【反思】

基础设施依赖要有降级路径，演示环境才能稳定答辩。

(4) 对 AI 辅助编程的思考（≥500 字）

AI 生成的代码是否都能理解？

不能全部逐行理解，尤其是 Prisma 聚合与推荐算法中的集合运算。处理方式是：让 AI 用注释说明输入输出；对核心路径（下单事务、支付、日志写入）人工走读一遍；用 Postman/脚本跑通后再合并。

是否有 AI 错误建议？如何识别？

有。例如曾建议使用不存在的 admin 订单接口、Vue2 的 `:visible.sync` 写法。识别方法：对照官方文档；以运行结果为准；对“大范围重写”保持警惕，优先小步 diff。

AI 会取代初级程序员吗？

短期内不会完全取代。AI 擅长模式化代码，但 **需求澄清、架构取舍、联调排错、安全与数据一致性** 仍需人负责。初级程序员若只会复制 AI 输出而不验证，反而容易被替代；若能驾驭 AI，效率会显著提升。

程序员的核心竞争力（新理解）

本次项目体会到核心竞争力在于：**问题分解、领域建模（电商+日志+分析）、质量意识（权限/事务/限流）、沟通与文档、持续验证**。工具会变，能交付可运行、可维护、可演示的系统的能力不会贬值。

下次如何更好利用 AI？

提前写好接口契约与数据表；让 AI 按模块生成但人工做 Code Review；每个迭代配 1~2 条自动化测试；报告类内容由 AI 起草后按评分标准删减；敏感配置永不写入仓库。

五、测试报告

5.1 测试方法

方法	应用范围
黑盒测试	按用户角色设计用例，不关注内部实现，验证输入输出
功能测试	对照需求 3.1~3.4 逐项验收
接口测试	Postman / <code>scripts/test-*.js</code>
安全测试	反爬虫脚本模拟异常 UA 与蜜罐访问

5.2 测试环境

项目	配置
前端地址	<code>http://localhost:3001</code>
后端地址	<code>http://localhost:3000</code>
数据库	PostgreSQL <code>shopping_db</code>
浏览器	Chrome 最新版

5.3 测试账号与口令

由 `node scripts/seed.js` 生成，密码均为 `123456`。

角色	账号	密码	说明
管理者	admin	123456	管理后台全部功能
销售人员	seller1~seller8	123456	卖家中心
普通用户	user1~user15	123456	购物与推荐

5.4 功能测试用例与结果

5.4.1 用户端

编号	用例	步骤	预期	结果
T-U01	用户注册	填写用户名密码注册	注册成功可登录	☑ 通过
T-U02	用户登录	user1 / 123456	跳转首页	☑ 通过
T-U03	浏览商品	打开首页与详情	列表与图片正常	☑ 通过
T-U04	加入购物车	详情页加购	购物车数量增加	☑ 通过
T-U05	下单支付	结算→支付	订单状态变 paid	☑ 通过（修复支付 Bug 后）
T-U06	浏览埋点	进入详情停留	BrowseBehaviorLog 有记录	☑ 通过
T-U07	推荐展示	详情页“也买了”	返回关联商品	☑ 通过

【图 5-1】用户首页商品列表（请插入截图）

【图 5-2】商品详情与推荐条（请插入截图）

【图 5-3】支付成功页面（请插入截图）

5.4.2 销售人员端

编号	用例	步骤	预期	结果
T-S01	卖家登录	seller1 / 123456	进入卖家中心	☑ 通过
T-S02	新增商品	填写名称价格库存	商品出现在列表	☑ 通过
T-S03	修改库存价格	编辑商品	保存成功	☑ 通过
T-S04	类别管理	新增/删除分类	分类同步更新	☑ 通过
T-S05	查看订单	打开订单列表	显示相关订单	☑ 通过

【图 5-4】卖家商品管理（请插入截图）

5.4.3 管理者端

编号	用例	步骤	预期	结果
T-A01	管理员登录	admin / 123456	进入仪表盘	☑ 通过
T-A02	添加卖家	新建 seller 账号	列表可见	☑ 通过
T-A03	重置密码	重置 seller 密码	新密码可登录	☑ 通过
T-A04	数据采集概览	打开采集页	四类计数与最近日志	☑ 通过
T-A05	用户画像	打开画像页	地域/购买力/偏好	☑ 通过
T-A06	销售趋势图	切换日/周/月	曲线与表格更新	☑ 通过
T-A07	销售异常	打开异常页	异常日期列表	☑ 通过
T-A08	销售排行	按销售额排序	Top 列表正确	☑ 通过
T-A09	反爬虫监控	运行 test 脚本后刷新	有拦截记录可解封	☑ 通过

【图 5-5】数据采集概览（请插入截图）

【图 5-6】用户画像（请插入截图）

【图 5-7】销售趋势图（请插入截图）

【图 5-8】反爬虫监控（请插入截图）

5.4.4 大数据与推荐专项

编号	功能点	验证方式	结果
T-B01	登录日志含 IP/时间	管理端采集页	☑ 通过
T-B02	浏览日志含类别/停留秒数	数据库 BrowseBehaviorLog	☑ 通过
T-B03	购买记录含单价/数量/日期	支付后查 PurchaseRecord	☑ 通过
T-B04	协同过滤推荐	登录 user1 看首页“猜你喜欢”	☑ 通过
T-B05	也买了推荐	商品详情底部推荐	☑ 通过

5.4.5 反爬虫专项

编号	用例	操作	预期	结果
T-C01	可疑 UA	python-requests 访问 /api/products	429 + needVerify	☑ 通过
T-C02	蜜罐	访问 /api/products/export-all-secret	403 + 记录 honeypot	☑ 通过
T-C03	人机验证	前端完成算术题	获得 Token 后可继续访问	☑ 通过
T-C04	管理解封	后台点击解封 IP	该 IP 可恢复访问	☑ 通过

执行命令：

```
cd back && node scripts/test-anti-crawler.js
```

5.5 测试指标汇总

指标	结果
功能用例总数	28
通过	28
失败	0
关键缺陷	支付日志 userId 问题（已修复）

5.6 在线部署验证

项目	填写
网站 URL	【请填写，如 <code>https://xxx.com</code> 】
管理账号	admin / 【密码】
用户账号	user1 / 【密码】
验证日期	【填写】
验证结论	☐ 功能完整可访问 ☐ 部分功能不可用

【图 5-9】线上首页访问截图（请插入截图）

六、附录

6.1 主要 API 列表

详见 [系统设计 - 3.4 接口设计](#)。

6.2 数据库迁移文件

- `back/prisma/migrations/` — 含用户角色、大数据采集表、反爬虫事件表等。

6.3 参考文献与资料

- Vue.js 官方文档：<https://cn.vuejs.org/>
- Express 官方文档：<https://expressjs.com/>
- Prisma 文档：<https://www.prisma.io/docs>
- Element Plus 文档：<https://element-plus.org/zh-CN/>
- 课程讲义《网络应用架构设计与开发》课程设计 PDF

6.4 提交检查清单

- ☐ 四份报告内容已按顺序合并在本文档
 - ☐ 已替换学号、姓名
 - ☐ 已插入关键页面截图（图 5-1～5-9）
 - ☐ 已填写代码仓库与在线 URL
 - ☐ 在线站点可访问至 2026-07-01
 - ☐ 文件名改为 学号_姓名.pdf 后提交学委
-

(报告完)